LangChain Interview Questions Set 1: Foundations, Prompts and the Chat Model Interface

Focus: Solid fundamentals, plus an understanding of why you would choose one approach over another. Candidates are not expected to have run large production systems, but they should be able to explain their choices clearly.

Code baseline: Python 3.12, LangChain 1.x (`langchain`, `langchain-core`, and provider packages such as `langchain-openai`). Where behaviour differs from older 0.x versions, the answer says so.

Topics covered in this set:

- What LangChain does and how its packages are organised
- Message types and prompt templates
- Creating chat models: `init_chat_model`, provider classes, model settings, API keys
- `invoke`, `batch`, `stream` and their async versions
- Reading responses: `content`, `text`, `response_metadata`, `usage_metadata`
- Structured output, rate limits and non-deterministic answers

Planned sets (junior track):

1. Foundations, prompts and the chat model interface (this file)
2. LCEL and Runnables
3. Memory and its types
4. Text splitters and retrievers
5. Hybrid and advanced RAG (BM25, ensemble retrieval, HyDE, multi-query, re-ranking)

Part 1: Questions

LangChain basics

Q1. Imagine a product manager asks you, "What does LangChain actually do for us, and what would we still have to build ourselves?" How would you answer?

Q2. You install `langchain`, then `from langchain_openai import ChatOpenAI` fails with a module-not-found error. Why, and what does this tell you about how LangChain is packaged?

Q3. You're building a maths tutor bot for school students. How would you decide what goes in the system message versus the human message, and what are AI messages for?

Prompt templates

Q4. A teammate builds prompts with plain Python f-strings inside their functions. Why might you suggest using LangChain prompt templates instead? Is there any case where an f-string is fine?

Q5. Write a prompt template that explains a topic at a chosen difficulty level. What does calling `invoke` on the template actually return, and how would you check what the model will receive?

Q6. Your summariser's output style is inconsistent. A colleague suggests adding three example summaries to the prompt. When does few-shot prompting help, and what does it cost?

Q7. Users paste Python code, which often contains curly braces, into a question that goes through your prompt template. Will that break the template? Explain your reasoning.

Creating and configuring chat models

Q8. You've seen code that uses `ChatOpenAI(...)` and other code that uses `init_chat_model("openai:...")`. When would you choose each?

Q9. You have two features: one classifies support tickets into five categories, and one writes marketing taglines. How would you set temperature, max tokens and timeout for each, and why?

Q10. How should API keys be handled in a LangChain project? What happens if the key is missing, and what mistakes have you seen people make?

invoke, batch and stream

Q11. Give me a realistic example of when you would use `invoke`, when you'd use `batch`, and when you'd use `stream`.

Q12. Write code that streams a model's answer to the terminal as it's generated. What is each chunk, and how would you get the complete message at the end?

Q13. When should you use `ainvoke` instead of `invoke`? What would go wrong if you used the wrong one in a web API?

Reading responses

Q14. You print a model response and see `AIMessage(content=..., response_metadata=..., usage_metadata=..., id=...)`. Walk me through those fields and tell me which ones you'd actually use in an app.

Q15. Your manager asks how much a new feature will cost per month before it launches. How would you use `usage_metadata` to estimate that?

Q16. You need the model to return a product name and price that your code can use, but sometimes it adds extra sentences around the data. What are your options, and which would you choose?

Errors and model behaviour

Q17. Your script loops over 500 reviews calling the model, and after a while it crashes with a `RateLimitError`. What does that mean, and how would you fix it?

Q18. A tester reports that the same question gives slightly different answers each time. Is that a bug? How would you explain it, and what can you do if consistency matters?

Part 2: Questions with Answers

Q1. Imagine a product manager asks you, "What does LangChain actually do for us, and what would we still have to build ourselves?" How would you answer?

I'd say LangChain is a toolkit that makes it easier to build applications on top of language models. It gives us one consistent way to talk to many different model providers, so switching from OpenAI to Anthropic is mostly a configuration change rather than a rewrite. It has ready-made pieces for common jobs, like prompt templates, loading and splitting documents, connecting to vector databases for search, and calling tools. It also gives us features like streaming, batching and tracing in a standard way.

What it doesn't do is decide what our product should do. We still have to design the prompts and test whether they give good answers, choose which model fits our cost and quality needs, prepare and clean our own data, build the user interface and the API, handle security, like who can see which data, and monitor quality and cost in production.

So I'd describe it as saving us from writing a lot of plumbing, while the parts that make our product good or bad are still our work. I'd also mention that for very simple use cases, like one prompt to one model, calling the provider's own SDK can be simpler, so it's a choice, not a requirement.

Q2. You install `langchain`, then `from langchain_openai import ChatOpenAI` fails with a module-not-found error. Why, and what does this tell you about how LangChain is packaged?

The error happens because the OpenAI integration is a separate package, `langchain-openai`, which isn't installed by `langchain` itself. The fix is:

```shell
pip install -U langchain langchain-openai
```

LangChain is split into several packages. `langchain-core` holds the basic building blocks that everything else depends on: messages, prompt templates and the common interface all components share. `langchain` contains higher-level features like `init_chat_model` and agents. Each major provider has its own package, like `langchain-openai` or `langchain-anthropic`, and there's `langchain-community` for many smaller integrations. In 1.x, older classes like the legacy chains moved to `langchain-classic`.

The reason for this split is that each provider's package can be updated on its own schedule when the provider releases new features, and you only install what you use, so the project has fewer dependencies and fewer conflicts.

Note that the package name uses a hyphen, `langchain-openai`, while the Python import uses an underscore, `langchain_openai`. That trips people up at first. I'd also pin versions in `requirements.txt` or `pyproject.toml`, because these packages update often.

Q3. You're building a maths tutor bot for school students. How would you decide what goes in the system message versus the human message, and what are AI messages for?

The system message is for instructions that apply to the whole conversation: who the assistant is and how it should behave. For the tutor, that would be something like: "You are a patient maths tutor for students aged 12 to 15. Guide the student step by step and don't just give the final answer. Use simple language." The student shouldn't need to repeat those rules, and they shouldn't be mixed into the student's question.

The human message is what the user actually says, such as "How do I solve 2x + 5 = 15?"

AI messages are the model's replies. When I build a multi-turn conversation, I send the earlier AI messages back along with the human messages, so the model knows what it already said. For example, if it already explained the first step, it can continue from there instead of starting over.

```python
from langchain.chat_models import init_chat_model
from langchain_core.messages import SystemMessage, HumanMessage, AIMessage

llm = init_chat_model("openai:gpt-4.1-mini")
messages = [
    SystemMessage("You are a patient maths tutor for students aged 12-15. Guide step by step."),
    HumanMessage("How do I solve 2x + 5 = 15?"),
    AIMessage("Good question! What could we do to both sides to remove the + 5?"),
    HumanMessage("Subtract 5?"),
]
print(llm.invoke(messages).text)
```

Keeping instructions in the system message makes the behaviour more consistent, and it's easier to change the rules in one place. One thing to remember is that system instructions guide the model, but they aren't a security guarantee; a user can still try to talk the model into ignoring them.

Q4. A teammate builds prompts with plain Python f-strings inside their functions. Why might you suggest using LangChain prompt templates instead? Is there any case where an f-string is fine?

Prompt templates give a few practical advantages. The prompt is defined once and separated from the logic, so it's easy to find, review and reuse. The template knows its required variables, so if I forget one, I get a clear error instead of a prompt with a blank in it. `ChatPromptTemplate` builds a proper list of messages with roles, which is what chat models expect, rather than one big string. Templates can be combined with models using the pipe operator, so `prompt | llm` works with streaming, batching and

tracing, and the formatted prompt shows up in traces when debugging. They also support things like pre-filling some variables with `partial`.

```python
from langchain_core.prompts import ChatPromptTemplate

review_prompt = ChatPromptTemplate.from_messages([
    ("system", "You review {language} code for bugs and readability."),
    ("human", "{code}"),
])
print(review_prompt.input_variables)   # ['code', 'language']
```

An f-string is fine for a quick experiment, a one-off script, or a very simple single call where none of those benefits matter. I wouldn't force templates where they add no value. But once a prompt is used in more than one place, needs testing, or is part of a chain, I'd move it into a template, because prompts change often, and having them in one clear place saves time.

Q5. Write a prompt template that explains a topic at a chosen difficulty level. What does calling `invoke` on the template actually return, and how would you check what the model will receive?

```python
from langchain_core.prompts import ChatPromptTemplate

explain_prompt = ChatPromptTemplate.from_messages([
    ("system", "You are a clear teacher. Explain topics for a {level} audience in under 150 words."),
    ("human", "Explain {topic}."),
])

prompt_value = explain_prompt.invoke({"topic": "recursion", "level": "beginner"})
print(type(prompt_value).__name__)   # ChatPromptValue
for m in prompt_value.to_messages():
    print(m.type, "->", m.content)
```

Calling `invoke` on the template doesn't call any model. It fills in the variables and returns a `ChatPromptValue`, which is basically the list of formatted messages. I can look at it with `to_messages()`, or `to_string()` if I want a single text version. That's my usual way of checking what the model will actually receive, because many bugs are just a variable that came through empty or with the wrong value.

There's also `ChatPromptTemplate.from_template("Explain {topic}.")`, which creates a template with a single human message. That's fine for quick cases, but I prefer `from_messages` when I need a system message, because it keeps the instructions separate from the user's input.

When I connect it with a model, `explain_prompt | llm`, the prompt value is passed straight to the model, so I don't need to convert anything myself. And in a real project, I'd write a small test that formats the prompt with sample values and checks the result, so changes to the template don't silently break it.

Q6. Your summariser's output style is inconsistent. A colleague suggests adding three example summaries to the prompt. When does few-shot prompting help, and what does it cost?

Few-shot prompting means including a few example inputs with the outputs I want. It helps most when the thing I want is easier to show than to describe, like a specific format, length, tone, or way of handling tricky cases. For a summariser, showing three summaries in the exact style I want, say one bold headline and three short bullet points, usually makes the output much more consistent than writing a long instruction about it.

It helps less for tasks where the model already behaves well with a clear instruction, and it can hurt if the examples are too similar to each other, because the model may copy their content or topics, not just their style. So I'd pick examples that vary in topic and length but share the same format.

The cost is tokens. The examples are sent with every request, so three long example summaries could double or triple the input size of each call, which increases cost and slightly increases latency. For summaries, the examples themselves include long input texts, so I'd use short example inputs.

I'd test it rather than assume. I'd run the same set of, say, twenty documents with and without examples and compare consistency and token usage. Sometimes a clearer instruction gets most of the benefit at a lower cost, and sometimes the examples are clearly worth it.

Q7. Users paste Python code, which often contains curly braces, into a question that goes through your prompt template. Will that break the template? Explain your reasoning.

No, it won't break. Curly braces only matter in the template text itself, which is the part I wrote when creating the template. That's where `{question}` is read as a placeholder. When I pass a value for `question`, the value is inserted as plain text, and it isn't treated as a template again, so braces inside the user's code are left alone.

```python
from langchain_core.prompts import ChatPromptTemplate

prompt = ChatPromptTemplate.from_messages([("human", "Explain this code:\n{code}")])
user_code = 'config = {"debug": True}\nprint(f"{config}")'
print(prompt.invoke({"code": user_code}).to_messages()[0].content)
```

That prints the code with its braces intact.

Where braces do cause problems is when I put them in the template text myself, for example writing a JSON example directly into the system message. Then LangChain thinks `{"debug": True}` is a variable and complains. The fix there is to double the braces, `{{` and `}}`, or pass the example in as a variable.

So the rule I remember is: braces in the template are syntax, braces in the values are just data. It's still worth testing with real user input, though, because very long pasted code can push the prompt over the model's context limit, which is a different problem.

Q8. You've seen code that uses `ChatOpenAI(...)` and other code that uses `init_chat_model("openai:...")`. When would you choose each?

`ChatOpenAI` is the provider-specific class from `langchain-openai`. `init_chat_model` is a helper in `langchain` that takes a provider and model name, like `"openai:gpt-4.1-mini"` or `"anthropic:claude-sonnet-4-5"`, and creates the right class for you. Both give you a chat model with the same methods, `invoke`, `stream` and so on.

```python
from langchain_openai import ChatOpenAI
from langchain.chat_models import init_chat_model

llm_a = ChatOpenAI(model="gpt-4.1-mini", temperature=0)
llm_b = init_chat_model("openai:gpt-4.1-mini", temperature=0)
```

I'd use `init_chat_model` when the model might change or come from configuration. For example, if the model name is in an environment variable or a settings file, I can switch providers without changing code, as long as the right provider package is installed. It's also what the LangChain docs use in most examples now.

I'd use the provider class directly when I'm committed to that provider and want its specific features with clear autocompletion and type hints in my editor. Some provider-specific options are easier to discover and configure on the specific class.

Either way, I'd create the model in one place in the codebase, like a small `get_llm()` function, rather than in every file. Then changing the model or its settings is one edit, and tests can swap in a fake model easily.

Q9. You have two features: one classifies support tickets into five categories, and one writes marketing taglines. How would you set temperature, max tokens and timeout for each, and why?

Temperature controls how random the model's word choices are. For the ticket classifier, I want the same ticket to get the same category every time, so I'd use a low temperature, 0 or close to it. For the taglines, variety is the point; I want different creative options, so I'd use a higher temperature, something like 0.8 to 1.0, and maybe ask for several options at once.

Max tokens limits how long the reply can be. A classifier only needs a category name, so a small limit, maybe 20 to 50 tokens, prevents rambling and keeps cost down. Taglines are short too, but if I ask for five options with short explanations, I'd allow a few hundred tokens. If the limit is too small, the answer gets cut off, which I can detect by checking the finish reason in `response_metadata`.

Timeout is how long I wait before giving up. A classifier running in the background can wait longer, while a user waiting in the UI shouldn't be stuck for a minute, so I'd set something like 20 to 30 seconds and show an error or retry.

```python
classifier_llm = init_chat_model("openai:gpt-4.1-mini", temperature=0, max_tokens=30, timeout=30)
tagline_llm = init_chat_model("openai:gpt-4.1-mini", temperature=0.9, max_tokens=300, timeout=20)
```

One thing to check: some reasoning models don't support changing temperature and may raise an error, so settings that work on one model don't always work on another. I'd test settings on the exact model

we deploy.

Q10. How should API keys be handled in a LangChain project? What happens if the key is missing, and what mistakes have you seen people make?

Keys should never be written in the code. LangChain's provider classes read standard environment variables automatically, like `OPENAI_API_KEY` or `ANTHROPIC_API_KEY`. For local development I keep them in a `.env` file loaded with `python-dotenv`, and make sure `.env` is in `.gitignore`. In production they come from the platform's secret manager, such as AWS Secrets Manager or the deployment's environment settings.

```python
from dotenv import load_dotenv
from langchain.chat_models import init_chat_model

load_dotenv()                       # reads OPENAI_API_KEY from .env
llm = init_chat_model("openai:gpt-4.1-mini")
```

If the key is missing, you get an error, usually when the model object is created or on the first call, saying the API key must be set. If the key is present but wrong or expired, you get an authentication error from the provider. It's better to check at startup and fail with a clear message than to fail on the first user request.

Common mistakes I've seen: committing the key to Git, even briefly, which means it must be rotated because it stays in the history; putting keys in notebooks that get shared; printing the full config or the environment in logs; sharing one personal key across a whole team, so nobody can track usage or revoke access cleanly; and putting the key in front-end code, where anyone can read it. The model should always be called from our backend, never directly from the browser.

Q11. Give me a realistic example of when you would use `invoke`, when you'd use `batch`, and when you'd use `stream`.

I'd use `invoke` when I need one complete answer and nobody is watching it being written. For example, an API endpoint that takes an email and returns a category, or a step in a script that generates a title for a document. I call it once and use the full result.

I'd use `batch` when I have many independent inputs to process the same way, like summarising 300 customer reviews or classifying all the tickets from yesterday. `batch` sends the requests in parallel, so it's much faster than calling `invoke` in a loop, and I can limit how many run at once with `max_concurrency` to avoid hitting rate limits.

```python
summaries = summarise_chain.batch(reviews, config={"max_concurrency": 5})
```

I'd use `stream` when a person is waiting and reading the answer, like a chatbot. The total time to generate the full answer is about the same, but the user starts seeing text in under a second instead of waiting several seconds for the whole thing, which feels much faster.

All three are available on every model and chain, and each has an async version, `ainvoke`, `abatch` and `astream`, which I'd use inside async code like a FastAPI endpoint.

Q12. Write code that streams a model's answer to the terminal as it's generated. What is each chunk, and how would you get the complete message at the end?

```python
from langchain.chat_models import init_chat_model

llm = init_chat_model("openai:gpt-4.1-mini")

full = None
for chunk in llm.stream("Explain what an API is in three sentences."):
    print(chunk.text, end="", flush=True)
    full = chunk if full is None else full + chunk

print()
print(full.text)             # the complete answer
print(full.usage_metadata)   # token counts, if the provider sent them
```

Each chunk is an `AIMessageChunk`, a small piece of the message, usually a few characters or a word. I print `chunk.text` with `end=""` so the pieces join on one line, and `flush=True` so they appear immediately instead of being buffered.

To get the complete message, I add the chunks together. `AIMessageChunk` supports the `+` operator, which joins the text and merges the other information, so at the end `full` is like the message I'd get from `invoke`. That's useful because I usually need the full answer afterwards, for example to save it to the conversation history.

Token usage information usually arrives with the last chunk, so I read it from the combined message rather than from individual chunks. With some providers and older package versions, you have to turn on usage reporting for streams, for example `stream_usage=True` on `ChatOpenAI`.

If I put a `StrOutputParser` after the model, the chunks become plain strings instead of message chunks, which is convenient when I only need the text.

Q13. When should you use `ainvoke` instead of `invoke`? What would go wrong if you used the wrong one in a web API?

`ainvoke` is the async version of `invoke`. I use it inside `async` code, like an `async def` endpoint in FastAPI, and I call it with `await`. While one request is waiting for the model's response, the server can handle other requests.

```python
from fastapi import FastAPI

app = FastAPI()

@app.get("/explain")
async def explain(topic: str):
    response = await llm.ainvoke(f"Explain {topic} simply.")
    return {"answer": response.text}
```

If I used plain `invoke` inside an `async def` endpoint, it would block. Model calls take seconds, and during that time the whole server's event loop is stuck, so other users' requests wait in line. With a few users it seems fine, but under real traffic everything gets slow. That's a hard bug to spot in testing.

The other mistake is calling `ainvoke` without `await`. That gives back a coroutine object instead of a response, and you get a confusing error when you try to read `.text`.

In a normal script or a Jupyter notebook running ordinary code, plain `invoke` is fine and simpler. Async helps when the code is already async and handles many things at once. If I really have to use sync code inside FastAPI, I can declare the endpoint with plain `def` instead of `async def`, and FastAPI runs it in a separate thread so it doesn't block.

Q14. You print a model response and see `AIMessage(content=..., response_metadata=..., usage_metadata=..., id=...)`. Walk me through those fields and tell me which ones you'd actually use in an app.

`content` is the model's reply. Usually it's a string, but with some providers and features, like Anthropic with tool use or extended thinking, it can be a list of content blocks. So I don't rely on it being a string.

`text` gives just the text of the reply as a string, even when `content` is a list of blocks. In LangChain 1.x it's a property, `response.text`. In older 0.3 versions it was a method, `response.text()`. This is what I use for displaying the answer.

`response_metadata` holds provider-specific details, like the exact model name and the finish reason. The finish reason tells me whether the model stopped naturally or was cut off by the token limit. The keys can differ between providers.

`usage_metadata` gives token counts in a standard format for all providers: `input_tokens`, `output_tokens` and `total_tokens`, sometimes with extra details like cached or reasoning tokens. I use it for tracking cost.

`id` is an identifier for the message, useful for logging and for managing message history.

There's also `tool_calls`, which contains any tool calls the model wants to make, if tools were provided.

```python
response = llm.invoke("Give me one tip for writing clean Python.")
print(response.text)
print(response.usage_metadata)
print(response.response_metadata.get("finish_reason"))   # key name varies by provider
```

In an app, I'd mainly use `text` for display, `usage_metadata` for cost logging, and the finish reason to detect truncated answers.

Q15. Your manager asks how much a new feature will cost per month before it launches. How would you use `usage_metadata` to estimate that?

I'd run the feature on a realistic sample, say 50 to 100 typical inputs, and record `usage_metadata` from each response. That gives me the average input tokens and output tokens per request. Using real samples matters, because prompt length depends on things like the system prompt, the user's text and any retrieved documents, and guessing usually underestimates it.

Then I'd look up the model's current price per million input and output tokens on the provider's pricing page, since output tokens usually cost more, and calculate:

```python
avg_in, avg_out = 1_800, 350           # measured from the sample
price_in, price_out = 0.40, 1.60       # example USD per million tokens; check current pricing
requests_per_month = 20_000 * 30       # expected requests per day x 30

cost_per_request = (avg_in * price_in + avg_out * price_out) / 1_000_000
monthly_cost = cost_per_request * requests_per_month
print(f"${cost_per_request:.5f} per request, about ${monthly_cost:,.0f} per month")
```

I'd present it as a range rather than one number, because traffic is an estimate, and if the feature is a chat, later turns include history, so they cost more than the first turn. I'd also mention which parts I could reduce if the cost is too high: a cheaper model, a shorter system prompt, fewer retrieved documents, or a lower output limit.

After launch, I'd keep logging `usage_metadata` for real traffic, so the estimate can be compared with the actual bill.

Q16. You need the model to return a product name and price that your code can use, but sometimes it adds extra sentences around the data. What are your options, and which would you choose?

There are a few options. I could ask for JSON in the prompt and parse it myself, but models sometimes add text like "Here is the JSON:" or wrap it in code formatting, so parsing fails now and then. I could use an output parser such as `JsonOutputParser`, which handles some of this, but it's still relying on the model following instructions.

The option I'd choose is `with_structured_output`. I describe the shape I want with a Pydantic model, and LangChain uses the provider's built-in features, like tool calling or JSON schema mode, to make the model return data in that shape. I get a Python object back, not text.

```python
from pydantic import BaseModel, Field
from langchain.chat_models import init_chat_model
```

```python
class Product(BaseModel):
    name: str = Field(description="Product name")
    price: float = Field(description="Price as a number, without currency symbol")

llm = init_chat_model("openai:gpt-4.1-mini", temperature=0)
extractor = llm.with_structured_output(Product)

product = extractor.invoke("The new AeroPress Go is on sale for Rs 2,999 this week.")
print(product.name, product.price)
```

The field descriptions help the model fill in values correctly, for example returning 2999 instead of "Rs 2,999".

It isn't perfect. The model can still get a value wrong or fail to produce valid data, especially if the text doesn't contain the information. So I'd make fields optional where the information might be missing, and handle errors, for example with `include_raw=True`, which returns the raw response and any parsing error instead of raising an exception.

Q17. Your script loops over 500 reviews calling the model, and after a while it crashes with a `RateLimitError`. What does that mean, and how would you fix it?

A rate limit error, an HTTP 429, means I'm sending requests faster than my account is allowed. Providers limit requests per minute and tokens per minute, and the limits depend on the account's tier. It doesn't mean my code is wrong; it means I need to slow down or retry.

I'd fix it in a few ways. First, retries: LangChain's chat models have a `max_retries` setting that retries failed requests with a wait in between, so a short burst over the limit recovers on its own. Second, control the speed: if I switch to `batch`, I set `max_concurrency` so only a few requests run at once, and LangChain's `InMemoryRateLimiter` can cap requests per second. Third, don't lose progress: I'd save results as I go, so if the script does crash at review 350, I can restart from there instead of from zero.

PYTHON

```python
from langchain_core.rate_limiters import InMemoryRateLimiter

limiter = InMemoryRateLimiter(requests_per_second=2)
llm = init_chat_model("openai:gpt-4.1-mini", max_retries=3, rate_limiter=limiter)

results = (prompt | llm).batch(reviews, config={"max_concurrency": 4}, return_exceptions=True)
```

`return_exceptions=True` means one failed review comes back as an error object instead of crashing the whole batch, so I can retry just the failures later.

I'd also check whether a different error is involved. An authentication error or a "context length exceeded" error won't be fixed by retrying, so I only retry errors that are temporary.

Q18. A tester reports that the same question gives slightly different answers each time. Is that a bug? How would you explain it, and what can you do if consistency matters?

It's usually not a bug. Language models generate text by choosing one token at a time from a list of likely options, and with a temperature above zero, there's some randomness in those choices. Different wording, different examples, even a different order of points, are all normal. Even with temperature 0, answers aren't always perfectly identical, because of how requests are processed on the provider's servers.

Whether it's a problem depends on the feature. For a chatbot, slight variation is fine and often feels more natural. For a classifier, a data extraction step, or anything that feeds into other code, I need consistent results.

If consistency matters, I'd set temperature to 0 or close to it, write a clearer prompt with explicit rules, and use structured output so the format is fixed even if wording varies. For classification, I'd restrict the output to a fixed list of categories, for example with an enum in the schema, so the model can't invent new labels. Some providers also support a `seed` parameter, which improves repeatability but is described as best effort, not a guarantee. If I need exactly the same answer for the same input, like for a common question, I can cache responses.

I'd also tell the tester what "correct" means for this feature. For variable text, tests should check the meaning or key facts, not exact wording, otherwise tests will fail randomly.